NumaRing: Topology-Aware Routing for NUMA-Local MPMC Queues, and What Broke When We Optimized It

Parth Kumar Sinha

Abstract—Cache-coherence traffic across NUMA sockets, not the choice of lock-free algorithm, is the dominant cost in a shared-counter multi-producer multi-consumer (MPMC) queue once threads span more than one socket. Per-node sharding plus batched cross-node transfer is a known pattern, not a new algorithm; this paper is a case study of building one such queue, NumaRing, and of two profiling-driven bugs and one textbook optimization that made it worse, all found by measuring rather than assuming. Caching a per-operation topology lookup that turned out to cost 183 ns (eleven to twenty times the ring-buffer operation it gated) cut that cost by 61× and raised low-contention throughput 6 to 7×. A second bug in the cross-node work-stealing path, a shared atomic that every thread touched on every overflow, cut median latency at 32 threads by 202-fold once fixed. CPU-pause backoff, the standard fix for a contended compare-and-swap retry loop, cost 17 to 30 percent of raw throughput under true sustained contention instead of helping; we measured it, reverted it, and report why. The evaluation runs on a two-socket, 32-vCPU cloud instance, the largest topology a fixed compute quota let us reach, and we caught our own cross-host measurement error before trusting a number from it. Raw throughput at 32 threads stayed far below the project's original design target, and nothing in the data says another round of fixes like these closes that gap on two sockets. Code, evaluation data, and the exact commits behind every before/after number in this paper are public.

Index Terms—NUMA, lock-free queue, MPMC, cache coherence, work stealing, tail latency, cache-conscious data structures

1 Introduction

A bounded MPMC queue built on a single shared head and tail counter works well inside one CPU socket. Every enqueue and dequeue is one compare-and-swap on that counter, and on a single-socket machine the counter's cache line stays close to every core touching it. That assumption breaks on a modern server CPU with two or more NUMA sockets. Once producer and consumer threads run on different sockets, every compare-and-swap on the shared counter forces the cache line holding it to migrate across the inter-socket interconnect. David et al. measured this directly: crossing a socket boundary costs two to 7.5 times what an intra-socket cache-line transfer costs, and that gap is a property of the hardware, not of any particular queue implementation [1].

NumaRing addresses this by giving each NUMA node its own queue instead of sharing one. A producer or consumer routes to the ring buffer on the node it happens to be running on, so the common case never touches another node's cache lines. When a node's local queue is temporarily full or empty, the queue moves a batch of items across nodes with one atomic claim instead of claiming items one at a time, which cuts the number of cross-socket compare-and-swap operations by roughly the batch size.

Building and evaluating that idea rigorously is most of this paper. Section 3 describes the design. Section 4 states the queue's semantics. Section 5 is the evaluation, and it is where the paper earns its keep: a routing-overhead bug we found by profiling and fixed for a 6 to 7× low-contention throughput gain, a work-stealing contention bug we found the same way and fixed for two to three orders of magnitude of tail-latency improvement at 32 threads, a backoff optimization we tried, measured, and reverted because it cost real throughput even though it looked like a win in one benchmark, and a methodology mistake (comparing two different cloud VM instances as if they were the same machine) that we caught before it became a false claim. Section 6 states plainly what the evaluation does not show: throughput at 32 threads is nowhere near the project's original target, and nothing in the data says another round of software fixes closes that gap. That ceiling is real, and a paper claiming otherwise without the hardware to back it up would not survive review at a systems venue.

Our contributions are: (1) a hierarchical, NUMA-aware MPMC queue design with batched cross-node work stealing, built on the bounded MPMC algorithm of Vyukov [2], itself a descendant of the Michael–Scott queue [3]; (2) a profiling-driven fix to a per-operation routing cost that we measured, not assumed, at 183 ns; (3) a second profiling-driven fix to work-stealing contention, evaluated with a same-host controlled methodology after we caught an earlier cross-host measurement mistake; (4) a negative result on CPU-pause backoff under sustained contention, included because it corrects a common assumption about backoff always helping; and (5) an honest accounting of what a 32-vCPU cloud quota does and does not let us claim.

• *P. K. Sinha is an independent researcher based in Oxford, United Kingdom. E-mail: sinhaparth555@gmail.com ORCID: 0009-0002-3120-9301*

2 Background and related work

Lock-free MPMC queues. Michael and Scott's non-blocking queue [3] established the two-pointer, compare-and-swap-based design most later concurrent queues build on. Vyukov's bounded MPMC queue [2] adapts that idea to a fixed-size ring buffer: every slot carries its own sequence number, so a producer or consumer only contends on the one slot it is about to touch, and the shared state is reduced to two atomic position counters. We use Vyukov's algorithm as the node-local ring buffer inside NumaRing (Section 3.1) for a specific reason: it is bounded and array-based, which is what a fixed-capacity, NUMA-pinned per-node buffer needs. Faster unbounded designs exist, LCRQ [4] chains CAS2-based segments to cut contention on x86, but an unbounded queue can grow its backing memory past the node it started on, which defeats the point of pinning it in the first place. `boost::lockfree::queue` [5] and moodycamel's `ConcurrentQueue` [6] are both mature, widely used C++ MPMC queues; neither is NUMA-topology-aware, and we use both as baselines in Section 5.

NUMA-aware synchronization. David et al. [1] is the closest thing to a definitive study of why synchronization stops scaling across sockets: they show that above a small number of threads, the bottleneck moves from the algorithm to the hardware cache-coherence protocol, and that this is true across lock-free, lock-based, and hybrid designs alike. Their result is the premise this paper starts from. We do not claim NumaRing escapes that hardware ceiling; Section 6 is explicit that it does not, at the scale we could test.

NUMA-aware locking. Lock cohorting [7] takes a different, complementary path to the same underlying problem: it turns an existing spin-lock algorithm into a NUMA-aware one by running a socket-local "cohort" lock per node and passing the lock between same-node threads before releasing it across sockets, cutting how often ownership of a shared lock crosses the interconnect. That is still fundamentally a single shared lock with node-local scheduling on top of it, one thread holds it at a time, everyone else waits. NumaRing does not have a shared lock at all in the common case: each node's queue is a fully independent instance, and cross-node traffic only exists on the batched work-stealing path when a node's own queue runs dry or fills up. The two approaches are not competing for the same problem so much as applying NUMA-awareness at different points in the design, cohorting to mutual-exclusion primitives, NumaRing to a specific concurrent data structure, and a data structure that does not need mutual exclusion at all is the more direct fix when the data structure itself, not a generic lock, is the bottleneck.

Detecting cross-socket contention. `perf c2c` [8] instruments the Linux performance-counter subsystem to report which cache lines suffer HITM (a load that hits a line held modified in another core's cache), the direct hardware signature of the problem this paper is about. We planned to use it. Section 5 explains why the cloud environment we tested on made that impossible, and what evidence we used instead.

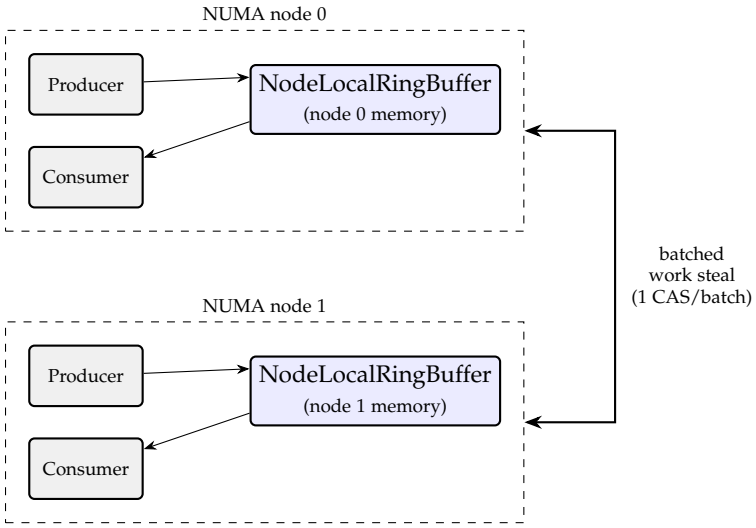

Fig. 1. NumaRing's two-node architecture. Each dashed box is one NUMA node's boundary. Producers and consumers route to their own node's ring buffer via `current_node()` (Section 3.2); the fast path never crosses the dashed boundary. Batched work stealing (Section 3.3) only runs when a node's ring is locally full or empty.

3 Design

NumaRing is a header-only C++20 library. Its top-level type, `Queue<T>`, owns one `NodeLocalRingBuffer<T>` per NUMA node reported by the host (Section 3.2) and adds a batched cross-node work-stealing layer on top (Section 3.3). On a single-node or non-NUMA host it degrades to exactly one ring buffer, so the design costs nothing on hardware that does not need it. Figure 1 shows the two-node case this paper evaluates.

3.1 Node-local ring buffer

Each node's queue is a bounded, lock-free ring buffer implementing Vyukov's algorithm [2]. Every slot holds a value and its own atomic sequence number. An enqueue reads the current tail position, checks the target slot's sequence number to confirm the slot is free, and claims the slot with one compare-and-swap on the tail counter; a dequeue is the mirror image on the head counter. Because the sequence number lives in the slot itself rather than in the shared counter, two threads only actually contend when they land on the same slot in the same generation, which under normal load is rare. This is the property that makes the algorithm fast on a single socket; it says nothing about what happens across sockets, which is the problem NumaRing's outer layer exists to address.

The ring's backing storage is allocated with `numa_alloc_onnode()` on the node the queue is constructed for, so the memory a node's threads touch on the fast path is local DRAM, not a cross-socket fetch. Both position counters and the per-slot sequence numbers are padded to 128 B (two cache lines) to prevent the L2 spatial prefetcher from pulling adjacent, independently-written lines into the same coherence unit, which would otherwise reintroduce false sharing that per-slot sequencing was supposed to eliminate. We did not measure false-sharing rates with and without this padding directly; that needs

perf c2c, which Section 5 explains was not available on our test hardware. The padding follows a well-established rule (match cache-line size, avoid straddling) rather than a measurement specific to this design.

3.2 Topology-aware routing

A producer or consumer thread finds its own ring buffer by asking which NUMA node it is currently running on (sched_getcpu() plus a CPU-to-node lookup) and indexing into the per-node array of ring buffers with the result. This call happens on every single try_enqueue and try_dequeue, so its cost is not incidental; Section 5.2 measures it directly and reports what we did about it. The alternative is a thread-local cached node id, refreshed only when the thread migrates; that trades this per-operation query for a small window where a just-migrated thread routes to its old node until the next refresh. We kept the per-operation query and its guaranteed freshness, and instead attacked its cost directly (Section 5.2), rather than accept a staleness window to avoid it.

3.3 Batched cross-node work stealing

When a thread's local ring buffer is full (on enqueue) or empty (on dequeue), Queue<T> does not fail immediately. It first tries to move a batch of items between the local ring and another node's ring, then retries the local operation. A batch transfer claims a contiguous run of slots with one compare-and-swap on each side instead of one compare-and-swap per item, so a batch of 16 costs 2 cross-node atomic operations instead of 32, an order of magnitude fewer touches on the line every other core sharing that ring buffer is also contending for. The single compare-and-swap that claims the whole contiguous range is the batch's linearization point: it either succeeds and every claimed slot becomes visible to other threads together, or it fails and none of them do, so a concurrent observer never sees a partial batch. If the batch transfer cannot move anything (every other node's ring is also full or also empty), Queue<T> falls back to a single-item attempt against every other node in turn before reporting failure to the caller, so correctness never depends on the fast batched path succeeding.

Section 5.3 describes two problems we found in this layer once we started measuring it at higher thread counts, and the fixes for both.

4 Correctness

Within one node's ring buffer, NumaRing inherits Vyukov's linearizability and FIFO-per-slot guarantees directly: the per-slot sequence number is the single point of agreement between producers and consumers, and every enqueue or dequeue that returns true corresponds to exactly one atomic transition of that number. A ring buffer's try_enqueue either succeeds and stores the value, or fails and leaves the caller's value completely untouched, which Queue<T>'s overflow handling depends on: a failed local enqueue can be retried against another node without a defensive copy.

Across nodes, Queue<T> does not offer a single global FIFO order, and does not claim to. An item enqueued on node 0 and later stolen to node 1 can be dequeued before an item enqueued slightly earlier that stayed on node 0 the whole time. This is a deliberate trade: a queue that enforced strict global ordering across sockets would need the same shared cross-socket counter this design exists to avoid. What NumaRing does guarantee is that a batch transfer is atomic from every other thread's point of view (a concurrent observer sees either the whole batch moved or none of it, never a partial batch), and that the fallback single-item path preserves the same per-slot sequencing guarantee the local ring buffer provides.

5 Evaluation

5.1 Setup and methodology

All numbers in this section come from a throwaway Google Compute Engine n2-standard-32 instance running Ubuntu 24.04, built fresh for each measurement round and deleted afterward. The instance exposes 32 vCPUs split across two NUMA nodes, 16 vCPUs each, with a measured node distance of 20 (versus 10 within a node), confirmed with numactl --hardware before every run. Each node's CPU list splits into two groups of eight offset by sixteen (node 0: CPUs 0–7 and 16–23), the standard numbering for simultaneous multithreading, so hyperthreading was on: 8 physical cores and 16 logical CPUs per node. This is a hard ceiling, not a choice: the GCP project used for this work has a 32-vCPU total quota across every region, and every machine type we found that reliably exposes more than two NUMA nodes needs more vCPUs than that. We built with GCC 13 at -O3 -march=native, against Boost 1.90.0 (Ubuntu 24.04's libboost-dev package, lockfree::queue built with fixed_sized<true> for a true bounded ring, matching NumaRing's own fixed capacity) and moodycamel's ConcurrentQueue at git tag v1.0.4, plus a queue built from std::mutex and std::deque as the traditional-locking baseline. NumaRing itself ran with its default configuration throughout: a ring capacity of 4096 slots per node and a work-stealing batch size of 16 items, neither varied in this evaluation. NumaRing is header-only C++20, Apache-2.0 licensed, and public at https://github.com/sinhaparth5/NumaRing (archived at https://doi.org/10.5281/zenodo.22232854); every before/after comparison in this paper is the same two commits, e33ef43 (before) and e8f0093 (after), built and run back to back on the same instance.

The workload pins one producer and one consumer to each NUMA node and has every producer enqueue a monotonic timestamp as its payload; a consumer's latency sample is the wall-clock time between that timestamp being written and the same value being dequeued by whichever thread gets it, which measures real cross-thread queueing delay rather than a same-thread round trip. All four implementations run the identical workload through the identical harness, with thread affinity set explicitly (numa_run_on_node()) rather than left to the scheduler, because a locality result is not meaningful if the thread-to-node mapping the queue is routing against is not itself stable.

Our first attempt at measuring the routing and work-stealing fixes compared a pre-fix build on one GCE instance against a post-fix build on a second, freshly created instance. Every implementation's numbers moved on

TABLE 1
Routing-lookup cost, measured directly on real 2-node NUMA hardware.

Metric	Before	After	Factor
`current_node()`	183 ns	2.97 ns	61.6×
`numa_node_of_cpu()`	130 ns	0.89 ns	146×

the second instance, including `boost::lockfree::queue` and `moodycamel::ConcurrentQueue`, neither of which our changes touch. A code change confined to `numaring::Queue` cannot make an unrelated library slower. The only explanation is that the two GCE instances landed on different physical hosts with different noisy-neighbor conditions, a known pitfall in cloud benchmarking. We caught this by checking whether libraries we had not touched moved too, before writing any comparison number down, and fixed it by checking out the pre-fix commit into a second build directory on the *same* instance (via a `git worktree`) and running both builds back to back. Every result below that compares a before and an after state used that same-host method.

5.2 Routing overhead

Profiling `current_node()`, the call every `try_enqueue` and `try_dequeue` makes to find its caller's local ring buffer, isolated the cost with two targeted microbenchmarks: `sched_getcpu()` alone costs 2.3–3.0 ns (it resolves through a vDSO call, not a real syscall trap), while `numa_node_of_cpu()` alone costs 130–131 ns, almost the entire measured cost of `current_node()`. It is a bitmask scan over every NUMA node that libnuma redoes from scratch on every call, despite the answer for a given CPU never changing for the life of the process. We replaced it with a lazily-built, process-wide lookup table from CPU to node, computed once and read thereafter; `sched_getcpu()` is still called fresh on every operation, so no staleness was introduced, only the static part of the lookup is cached.

For context, a single-threaded enqueue-dequeue round trip through the underlying ring buffer, with no routing call at all, costs 16.2 ns. Before this fix, routing alone cost more than eleven times that; after, it costs less than a fifth of it. That 61× is the cost of one call in isolation; Table 2 in the next section reports what removing that cost did to end-to-end 2-thread throughput (roughly 6 to 7×), a different measurement of the same fix, not the same number restated.

5.3 Work-stealing contention

The routing fix alone left a puzzle: it delivered a large gain at 2–8 threads and almost none at 16–32, where tail latency stayed in the millisecond range. The shape of that result, a fast median next to a slow p99 and p99.9, is the signature of a shared contended resource rather than raw per-operation cost, so we looked for one in the work-stealing path and found two.

First, candidate selection for which other node to steal from or spill to went through a single `std::atomic<std::size_t>` that every thread incremented on every overflow or underflow event. Under saturation, when most producers on most nodes are overflowing at once, that counter becomes a cache line contended by every thread on the fast-failure path, for no benefit beyond spreading out which candidate node gets tried first. We replaced it with per-thread state, seeded once per thread and advanced independently, which needs no cross-thread synchronization at all.

Second, a steal attempt would proceed against any candidate node with at least one item available, which under saturation meant several threads simultaneously racing to steal the last few items from a nearly empty source, most of their compare-and-swap attempts doomed to fail and retry. We added a cheap, deliberately approximate size check (two relaxed atomic loads, never used for a correctness decision) and gated a steal attempt on the source having at least twice the batch size available. A skipped attempt falls back to the existing single-item cross-node path, so this only changes which code path handles the low-occupancy case, never correctness.

Figure 2 plots throughput against thread count for the fixed build against all three baselines, on the same host, same run. Figure 3 plots p50/p95/p99/p99.9 latency for the same four implementations at 32 threads.

Table 2 and Figure 2 both come from the latency-tracking harness described in Section 5.1, which calls the system clock around every operation to build the percentiles in Table 3. That per-operation clock call is not free, and it is why the 32-thread throughput here (0.321 M items/s) looks nothing like the throughput reported later from an uninstrumented driver with no clock call at all (Section 5.5, Table 4: 9.70 M ops/s for the same build). Both numbers are real; they answer different questions, and neither should be read as a correction of the other.

Table 3 is worth reading carefully rather than skimming

TABLE 2
`numaring::Queue` throughput and median latency, same host, before and after both routing and work-stealing fixes.

Threads	Throughput (M ops/s) Before	After	p50 latency Before	After
2	1.73	12.35	3480 µs	218 µs
32	0.260	0.321	1690 µs	8.4 µs

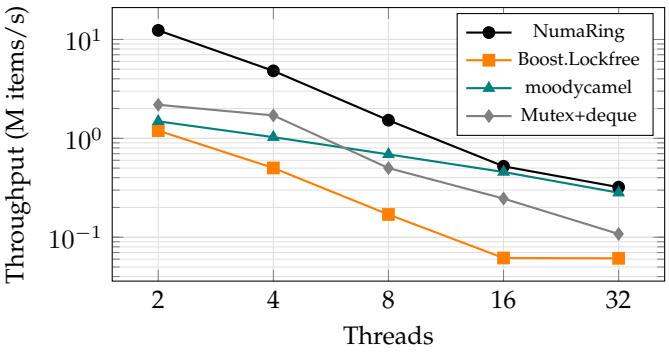

Fig. 2. Throughput vs. thread count, same host, current build. All four implementations run the identical timestamp-payload workload through the same latency-tracking harness (a per-operation clock call, not a raw-throughput driver); Section 5.5 reports the much higher uninstrumented rate for the same NumaRing build.

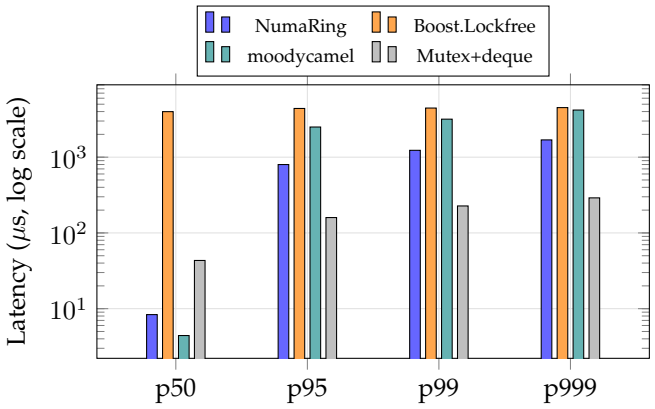

Fig. 3. p50/p95/p99/p99.9 latency at 32 threads, same host, current build. NumaRing has the lowest median; the mutex baseline (gray) has the lowest p99 and p99.9, the opposite ranking, at this thread count. See Table 3 for the exact values plotted.

TABLE 3
Latency percentiles at 32 threads, same host, current build.

Implementation	p50	p95	p99	p99.9
NumaRing	8.4 µs	799 µs	1230 µs	1690 µs
Boost.Lockfree	3990 µs	4400 µs	4450 µs	4510 µs
moodycamel	4.42 µs	2500 µs	3170 µs	4190 µs
Mutex+deque	43.3 µs	160 µs	227 µs	290 µs

for a winner. NumaRing's median is the lowest of the four at 32 threads, but its tail is not: p99 and p99.9 land between Boost's and moodycamel's, both well above the mutex baseline's tail at this specific thread count and workload. We are not going to round that up. A plausible reason the blocking mutex wins on tail latency here: a thread blocked on a contended mutex is descheduled, so it stops competing for the core's execution ports and cache bandwidth until the lock is free, while every lock-free design in this comparison keeps a losing thread spinning and retrying compare-and-swap attempts, which adds contention exactly when the queue is already saturated. We have not instrumented this directly, so we state it as a plausible explanation, not a measured one. Section 6 says what we think is actually going on.

5.4 NUMA memory locality

`numastat`, sampled before and after a 15-second sustained run at 32 threads, showed `numa_hit`/`numa_miss` ratios of 100% on both nodes, with zero misses recorded on either. A per-process snapshot taken mid-run (`numastat -p`) showed private memory split across both nodes rather than concentrated on one, consistent with one `NodeLocalRingBuffer` correctly landing on each node's own memory via `numa_alloc_onnode()`. Because `NodeLocalRingBuffer` allocates its backing storage once at construction and never again during steady-state operation, the system-wide counters move only slightly during a run; that is expected given the design, not a measurement gap.

TABLE 4
`numaring_sustained_load` throughput (M ops/s) at 32 threads, same host, 15-second runs, three repeats per variant.

Variant	Run 1	Run 2	Run 3	Mean
Pre-fix (no backoff)	8.65	9.41	9.02	9.03
Flat pause	7.44	8.08	6.95	7.49
Exponential backoff	7.25	5.44	6.47	6.39
Work-stealing fix, no backoff	9.82	8.24	11.03	**9.70**

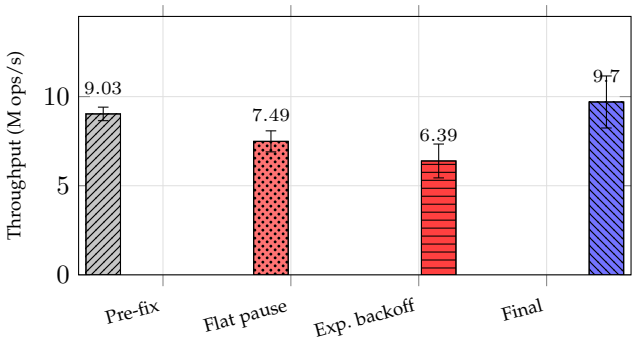

Fig. 4. `numaring_sustained_load` throughput at 32 threads, same host (Table 4): mean of three 15-second runs, error bars span the observed range. Both backoff variants (red) sit below the pre-fix baseline (gray); the work-stealing fix with no backoff (blue) is the only variant that improves on it.

5.5 A negative result: CPU-pause backoff

Standard advice for a contended compare-and-swap retry loop is to back off: pause briefly before retrying, so a losing thread does not immediately re-contend the same cache line. We implemented two variants in the ring buffer's own retry loops, a flat `__builtin_ia32_pause`-per-retry version and a version that grows the pause geometrically with consecutive failures up to a bound, and measured both against an uninstrumented, fixed-duration sustained-load driver at 32 threads that has no per-operation clock call diluting the measurement.

The 9.70 M ops/s in Table 4 is the whole 32-thread system, enqueues and dequeues counted together (every successful `try_enqueue` or `try_dequeue` is one op, matching how the other tables in this paper count); per thread that is roughly 303,000 ops/s. The 0.321 M items/s instrumented figure in Table 2 is around 10,000 items/s per thread by the same accounting, the clock call being the difference described above.

Both backoff variants reduced raw throughput, and the more sophisticated one reduced it further, not less. Under this driver's true wall-to-wall contention, with no pacing between a thread's own retries, the fastest path back to success for a losing compare-and-swap attempt is to retry immediately; the atomic read-modify-write itself already serializes access through the cache coherence protocol, so spending cycles on a pause instruction is pure loss, and a growing pause loses more. We had measured the same backoff variants *helping* tail latency in the instrumented benchmark from Section 5.3, which uses a real clock call around every operation; our best explanation for the discrepancy is that the clock call itself inserts wall-clock spacing between a thread's successive

retries that the uninstrumented driver does not have, and that spacing is enough to make a fixed pause look free when it is not. We reverted both backoff variants and report this result because it corrects an assumption (that backoff helps a retry loop) that does not hold under every workload shape, and because a paper that only reports what worked would be reporting half the evidence.

5.6 What we could not measure

`perf c2c` needs the host to expose a hardware performance-monitoring unit with support for precise memory-load sampling. `perf stat -e cycles,instructions` on our test instance reported every counter, including plain cycle counting, as `<not supported>`; the kernel log confirmed the reason directly: "`Performance Events: unsupported CPU family 6 model 85 no PMU driver, software events only.`" GCP has a `--performance-monitoring-unit=enhanced` flag for exactly this, but it was rejected for every machine type we could reach under our 32-vCPU quota, including both `n2-standard-32` and `c3-standard-22`. We are reporting this as a limitation rather than omitting it, because a paper that is honest about what its evaluation shows should also be honest about what it does not.

6 Discussion and limitations

The evaluation supports two conclusions that both matter and point in different directions.

The routing fix and the work-stealing fix are real, and the tail-latency numbers behind them are not close calls: a 202-fold drop in median latency and a 3.1-fold drop in p99 at 32 threads, on the same host, measured before and after with nothing else changed. Anyone building a NUMA-local queue on a hot path that queries thread affinity per operation should expect the same class of bug we found, because the cost of a topology query is easy to assume is negligible and expensive to verify is not.

The throughput ceiling is also real, and no amount of software tuning we tried moved it much. At 32 threads across two sockets, NumaRing's raw throughput sits in the low hundreds of thousands of operations per second in the instrumented benchmark and around ten million in the uninstrumented driver, both far short of the workload's original 120 million operations per second target. That number is worth being honest about: it comes from this project's own design roadmap, set before any of the evaluation in this paper, not from a derivation, a reference machine, or a competing system's published number, and we cannot reconstruct after the fact what it was based on. We report the miss against it because it was the stated goal, not because we can defend the number itself. We do not think a third round of the same kind of fix closes that gap. The two fixes in this paper targeted software overhead this project's own code introduced (a redundant lookup, an unnecessary shared atomic); once those are gone, what remains is the same hardware limit the motivation in Section 5.1 describes: two sockets connected by an interconnect with fixed bandwidth and fixed latency, and a workload that deliberately routes half its traffic across that interconnect. Closing that gap, if it can be closed at all with this design, needs either a larger NUMA topology than two sockets to test against, or a different approach to how the shared position counters are contended, and we do not have evidence for either from a 32-vCPU cloud quota.

The evaluation's other limits are worth stating plainly rather than leaving implicit. Every number in Section 5 comes from one machine type (`n2-standard-32`) in one cloud region, and while we controlled for host-to-host variance within that constraint, we have not tested a four- or eight-socket topology, a bare-metal two-socket machine with more cores per node, a different interconnect, or a different cloud provider. Two nodes is also the smallest topology where a hierarchical, per-node design can show any effect at all; we cannot tell from this evaluation whether batched work stealing pays off the same way, better, or worse once there are more than two nodes to steal between. The workload is a synthetic, symmetric producer-consumer benchmark with a fixed capacity and batch size; it is not a trace from a real application, and different workload shapes could move the work-stealing fix's benefit up or down. `perf c2c` confirmation of the cross-socket cache-invalidation reduction this design is meant to deliver remains unmeasured, for the environmental reason given in Section 5, not because we chose not to look.

7 Conclusion

We built a NUMA-aware MPMC queue, found and fixed two real performance bugs by profiling rather than guessing, tried an optimization that looked correct and measured it costing throughput instead of saving it, caught a benchmarking mistake before it became a false claim, and are reporting a throughput target we did not hit alongside the latency improvements we did. We think that is a more useful contribution than a set of numbers tuned to look better than the hardware we tested on actually supports.

References

[1] T. David, R. Guerraoui, and V. Trigonakis, "Everything you always wanted to know about synchronization but were afraid to ask," in *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles (SOSP)*, 2013, pp. 33–48.

[2] D. Vyukov, "Bounded mpmc queue," https://www.1024cores.net/home/lock-free-algorithms/queues/bounded-mpmc-queue, 2010.

[3] M. M. Michael and M. L. Scott, "Simple, fast, and practical non-blocking and blocking concurrent queue algorithms," in *Proceedings of the Fifteenth Annual ACM Symposium on Principles of Distributed Computing (PODC)*, 1996, pp. 267–275.

[4] A. Morrison and Y. Afek, "Fast concurrent queues for x86 processors," in *Proceedings of the 18th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, 2013, pp. 103–112.

[5] Boost C++ Libraries, "Boost.lockfree," https://www.boost.org/doc/libs/release/doc/html/lockfree.html, 2024.

[6] C. Desrochers, "A fast general purpose lock-free queue for C++," https://moodycamel.com/blog/2014/a-fast-general-purpose-lock-free-queue-for-c++, 2014.

[7] D. Dice, V. J. Marathe, and N. Shavit, "Lock cohorting: A general technique for designing NUMA locks," in *Proceedings of the 17th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, 2012, pp. 247–256.

[8] J. Mario, "C2c – false sharing detection in Linux perf," https://joemario.github.io/blog/2016/09/01/c2c-blog/, 2016.